

ENSF 380

TOPIC 02: OBJECT-ORIENTED PROGRAMMING PART I

What is OOP?

Software design is organized around data rather than functions

Data and the methods which operate on it are collected together

Definitions

Function: A reusable block of code that performs a task, and often takes input values and returns output.

Method: A function which is associated with an object; it can access or modify object properties.

Class: A template which defines the properties, methods, and behaviors of objects.

Object: An instantiation of a class which has values for properties and methods.

Data Structures are Like Objects

Key	Value
shape	circle
radius	3
colour	green



This dictionary (and every data structure of type dictionary)

```
shapedict.get("shape") # get value for key "shape"
```

```
shapedict.keys() # get a list of all keys
```

```
shapedict.items() # get a list of key:value pairs
```



...can perform actions on the data contained within

Which is Better for Cooking?



Photo Credits:

Sarah Chai - <https://www.pexels.com/photo/kitchenware-on-shelves-in-kitchen-7262728/>

Jason Wun - <https://www.flickr.com/photos/25pics/2444206000>

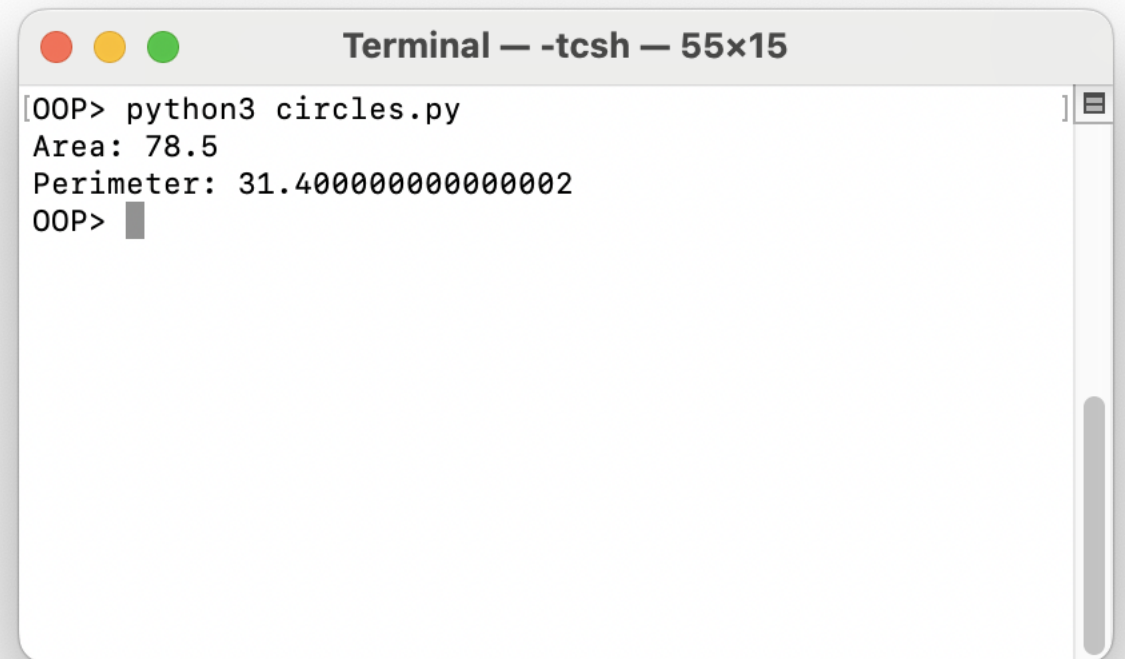
Python Example: Circle

01_PythonCircles

Program

```
def area(radius):  
    return 3.14 * radius ** 2  
  
def perimeter(radius):  
    return 2 * 3.14 * radius  
  
radius = 5  
print("Area:", area(radius))  
print("Perimeter:", perimeter(radius))
```

Output

A terminal window titled "Terminal — -tcsh — 55x15" with standard macOS window controls (red, yellow, green buttons). The terminal shows the execution of a Python script. The prompt is "[OOP>". The command entered is "python3 circles.py". The output is "Area: 78.5" followed by "Perimeter: 31.400000000000002". The prompt returns to "[OOP>".

```
[OOP> python3 circles.py  
Area: 78.5  
Perimeter: 31.400000000000002  
[OOP> █
```

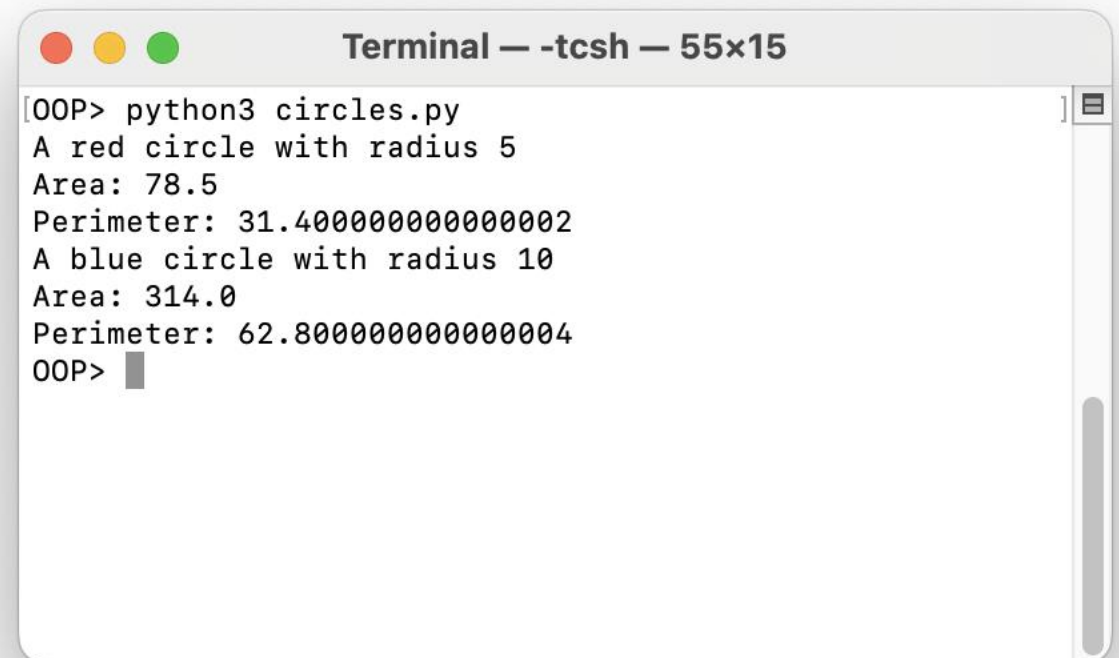
Python Example: Circles

02_PythonCircles

Program

```
... def describe(radius, colour):  
+     return f"A {colour} circle with radius  
    {radius}"  
  
circle1_radius = 5  
circle1_colour = "red"  
circle2_radius = 10  
circle2_colour = "blue"  
  
print(describe(circle1_radius, circle1_colour))  
print("Area:", area(circle1_radius))  
print("Perimeter:", perimeter(circle1_radius))  
  
print(describe(circle2_radius, circle2_colour))  
print("Area:", area(circle2_radius))  
print("Perimeter:", perimeter(circle2_radius))
```

Output



A terminal window titled "Terminal — -tcsh — 55x15" displays the output of running a Python script. The output shows the description and calculated area and perimeter for two circles: a red circle with radius 5 and a blue circle with radius 10. The perimeter values are displayed with high precision, showing trailing zeros.

```
[OOP> python3 circles.py  
A red circle with radius 5  
Area: 78.5  
Perimeter: 31.400000000000002  
A blue circle with radius 10  
Area: 314.0  
Perimeter: 62.800000000000004  
OOP> █
```

Python Example: Many Circles

03_PythonCircles

Functions

```
...  
def area(radius, units):  
    return f"{3.14 * radius ** 2} {units}^2"  
  
def perimeter(radius, units):  
    return f"{2 * 3.14 * radius} {units}"  
  
def describe(radius, colour, material):  
    return f"A {colour} circle made of {material}  
+    with a radius of {radius}"  
...
```

Definition of circles

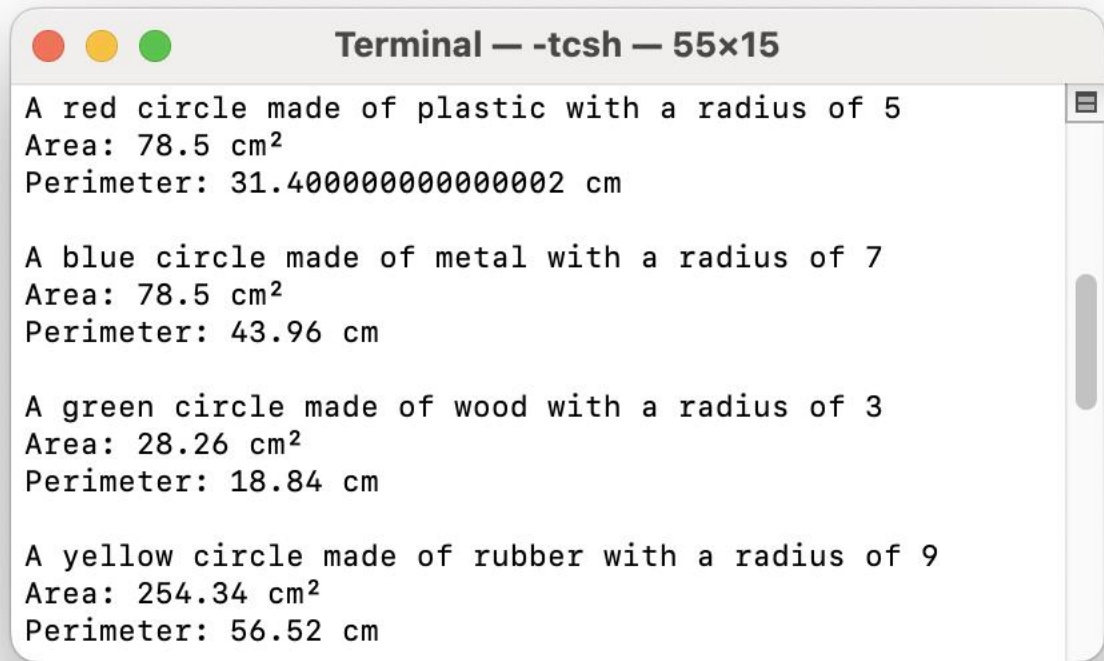
```
...  
circle1_radius = 5  
circle1_colour = "red"  
circle1_material = "plastic"  
circle1_units = "cm"  
  
circle2_radius = 7  
circle2_colour = "blue"  
circle2_material = "metal"  
circle2_units = "cm"  
...
```


Python Example: Many Circles

03_PythonCircles

Output

```
... print(describe(circle1_radius, circle1_colour, circle1_material))
print("Area:", area(circle1_radius, circle1_
print("Perimeter:", perimeter(circle1_radius,
print()
print(describe(circle2_radius, circle2_colour, circle2_material))
print("Area:", area(circle1_radius, circle2_
print("Perimeter:", perimeter(circle2_radius,
print()
...
```



A terminal window titled "Terminal — -tcsh — 55x15" displays the output of the Python program. The output consists of four lines of text, each describing a circle with its radius, area, and perimeter. The circles are: a red circle made of plastic with a radius of 5, a blue circle made of metal with a radius of 7, a green circle made of wood with a radius of 3, and a yellow circle made of rubber with a radius of 9. The area is calculated in cm² and the perimeter is calculated in cm.

```
A red circle made of plastic with a radius of 5
Area: 78.5 cm²
Perimeter: 31.400000000000002 cm

A blue circle made of metal with a radius of 7
Area: 78.5 cm²
Perimeter: 43.96 cm

A green circle made of wood with a radius of 3
Area: 28.26 cm²
Perimeter: 18.84 cm

A yellow circle made of rubber with a radius of 9
Area: 254.34 cm²
Perimeter: 56.52 cm
```

Python Example: Many Circles (Arrays)

04_PythonCircles

```
...  
+  
+  
# Define arrays for circle attributes  
radii = [5, 7, 3, 9, 6, 4, 8, 10, 2, 12]  
colours = ["red", "blue", "green", "yellow", "purple", "orange", "pink", "white", "black", "brown"]  
materials = ["plastic", "metal", "wood", "rubber", "glass", "ceramic", "stone", "paper", "foam", "leather"]  
units = ["cm"] * 10 # All circles use the same units  
  
# Print details for each circle  
print("Intermediate Version with Arrays for Attributes:")  
for i in range(len(radii)):  
    print(describe(radii[i], colours[i], materials[i]))  
    print("Area:", area(radii[i], units[i]))  
    print("Perimeter:", perimeter(radii[i], units[i]))  
    print()
```

Python Example: Many Circles (Dictionaries)

05_PythonCircles

```
...
circle1 = {"radius": 5, "colour": "red", "material": "plastic", "units": "cm"}
circle2 = {"radius": 7, "colour": "blue", "material": "metal", "units": "cm"}
circle3 = {"radius": 3, "colour": "green", "material": "wood", "units": "cm"}
circle4 = {"radius": 9, "colour": "yellow", "material": "rubber", "units": "cm"}
...
circle5 = {"radius": 6, "colour": "purple", "material": "glass", "units": "cm"}

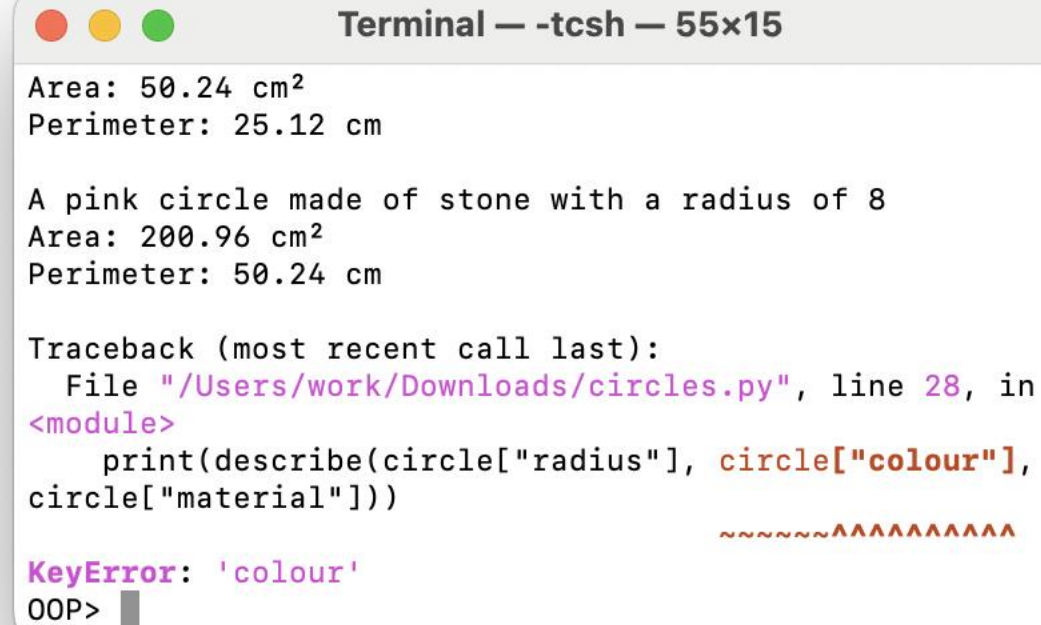
circles = [circle1, circle2, circle3, circle4, circle5, circle6, circle7, circle8, circle9, circle10]

# Print details for each circle
for circle in circles:
    print(describe(circle["radius"], circle["colour"], circle["material"]))
    print("Area:", area(circle["radius"], circle["units"]))
    print("Perimeter:", perimeter(circle["radius"], circle["units"]))
    print()
```

Python Example: Many Circles (Dictionaries)

05_PythonCircles

```
...
circle7 = {"radius": 8, "colour": "pink", "material": "stone", "units": "cm"}
circle8 = {"radius": 10, "color": "white", "material": "paper", "units": "cm"}
circle9 = {"radius": 2, "colour": "black", "material": "foam", "units": "cm"}
...
```



Terminal — -tcsh — 55x15

```
Area: 50.24 cm2
Perimeter: 25.12 cm

A pink circle made of stone with a radius of 8
Area: 200.96 cm2
Perimeter: 50.24 cm

Traceback (most recent call last):
  File "/Users/work/Downloads/circles.py", line 28, in
<module>
    print(describe(circle["radius"], circle["colour"],
circle["material"]))
                                             ~~~~~^~~~~~
KeyError: 'colour'
OOP> █
```

Python Example: More Shapes

```
def circle_area(radius):  
    return 3.14 * radius ** 2  
  
def circle_perimeter(radius):  
    return 2 * 3.14 * radius  
  
def describe_circle(radius, colour):  
    return f"A {colour} circle with radius {radius}"  
  
def square_area(side):  
    return side ** 2  
  
def square_perimeter(side):  
    return 4 * side  
  
def describe_square(side, colour):  
    return f"A {colour} square with side length {side}"
```

...

OOP Benefits

Related data is stored together (encapsulation)

Consistency of available attributes for all data of the same type

Links between functions and the data they act upon (encapsulation)

Distance from implementation details (abstraction)

Consistency between different but related objects (inheritance)

Cornerstones of OOP

Encapsulation

Abstraction

Inheritance

Polymorphism

Python OOP Example: Circles

07_OOP_PythonCircles

```
class Circle:
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2
    def perimeter(self):
        return 2 * 3.14 * self.radius

thecircle = Circle(5)
print("Area:", thecircle.area())
print("Perimeter:", thecircle.perimeter())
```


Python Examples: Circles

Without OOP

01_PythonCircles

```
def area(radius):  
    return 3.14 * radius ** 2  
  
def perimeter(radius):  
    return 2 * 3.14 * radius  
  
radius = 5  
print("Area:", area(radius))  
print("Perimeter:", perimeter(radius))
```

With OOP

07_OOP_PythonCircles

```
class Circle:  
    def __init__(self, radius):  
        self.radius = radius  
    def area(self):  
        return 3.14 * self.radius ** 2  
    def perimeter(self):  
        return 2 * 3.14 * self.radius  
  
thecircle = Circle(5)  
print("Area:", thecircle.area())  
print("Perimeter:", thecircle.perimeter())
```

Python OOP Example: Multiple Circles

08_OOP_PythonCircles

```
class Circle:
    def __init__(self, radius, colour):
        self.radius = radius
        self.colour = colour

    def area(self):
        return 3.14 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14 * self.radius

    def describe(self):
        return f"A {self.colour} circle with radius {self.radius}"
```

...

Python OOP Example: Multiple Circles

08_OOP_PythonCircles

...

```
circle1 = Circle(5, "red")
circle2 = Circle(10, "blue")

print("OOP Example:")
print(circle1.describe())
print("Area:", circle1.area())
print("Perimeter:", circle1.perimeter())

print(circle2.describe())
print("Area:", circle2.area())
print("Perimeter:", circle2.perimeter())
```

Python OOP Example: More Shapes

09_OOP_PythonSquares

...

```
class Square:
    def __init__(self, radius, colour):
        self.radius = radius
        self.colour = colour

    def area(self):
        return self.radius ** 2

    def perimeter(self):
        return 4 * self.radius

    def describe(self):
        return f"A {self.colour} square with side length {self.radius}"
```

...

Python OOP Example: More Shapes

09_OOP_PythonSquares

...

```
# Create instances of Circle and Square
circle = Circle(5, "red")
square = Square(4, "blue")

print("OOP Example:")
print(circle.describe())
print("Area:", circle.area())
print("Perimeter:", circle.perimeter())

print(square.describe())
print("Area:", square.area())
print("Perimeter:", square.perimeter())
```

Python OOP Example: More Shapes

10_OOP_PythonSquares

...

```
class Square:
    def __init__(self, side, colour):
        self.side = side
        self.colour = colour

    def area(self):
        return self.side ** 2

    def perimeter(self):
        return 4 * self.side

    def describe(self):
        return f"A {self.colour} square with side length {self.side}"
```

...

```
square = Square(4, "blue")
```

...

```
print(square.describe())
print("Area:", square.area())
print("Perimeter:", square.perimeter())
```

SOLID: Better OO Design

S – Single Responsibility Principle (SRP)

O – Open/Closed Principle (OCP)

L – Liskov Substitution Principle (LSP)

I – Interface Segregation Principle (ISP)

D – Dependency Inversion Principle (DIP)

Single Responsibility Principle

Each class should have one reason to change

Focus on a single responsibility or function

Advantages

- Easier to test and maintain
- Reduces coupling between responsibilities

Example

- A Report class should handle the content and structure of the Report, not printing or saving the report
- Printing and saving the report are handled by other classes

Open/Closed Principle

Software entities should be open for extension, but closed for modification

Modify behavior by adding new code, not altering existing code

Advantages

- Avoids breaking existing functionality
- Simplifies adding features

Example

- A class `PaymentProcessor` has a method `processPayment()` which accepts a `PaymentMethod` object as a parameter
- It calls `process()` on that object
- `PaymentMethod` is an interface which defines a contract for a method `process()`
- Individual payment methods (e.g., `ApplePay`) are implemented as classes which realize the `PaymentMethod` interface (e.g., `ApplePayPayment`) and provide the `process()` method
- The design is "open for extension" because new payment methods can be added
- It is "closed for modification" because adding a new payment method doesn't require modifying `PaymentProcessor`

Liskov Substitution Principle

Subtypes must be replaceable for their base types without altering behavior

Advantages

- Ensures consistent behavior across inheritance hierarchies
- Avoids unexpected runtime errors

Example

- A company has an Employee class which is inherited by different roles, such as Sales and HumanResources
- Each child class provides a different bonus calculation in the form of a calculateBonus() method
- The company also has Contractor which does not provide a calculateBonus() method
- The Contractor class should not inherit from the Employee class because it does not provide the expected functionality of the Employee class

Interface Segregation Principle

Prefer multiple small, specific interfaces over large, general ones

Advantages

- Prevents implementing unnecessary methods
- Reduces code dependency on irrelevant details

Example

- A company produces robot vacuums and robot mop-vacuums
- Instead of a Robot interface with methods for features such as vacuuming and mopping, this functionality should be split into smaller, specific interfaces so that each robot class only needs to implement the interfaces it needs

Dependency Inversion Principle

Depend on abstractions, not concrete implementations

Advantages

- Reduces coupling between high- and low-level modules
- Simplifies testing and swapping implementations

Example

- If a Report class instantiates and uses a Database class, there will be a tight coupling between them
- Instead, Database should be an interface which can be realized by classes such as PostgreSQLDatabase and MySQLDatabase
- The Report class should receive a Database object via the constructor or setter and can call a defined method such as retrieveSales() on the object

Summary: Knowledge

Definitions of function, method, class and object

Advantages of object-oriented programming

The five SOLID principles